

Test Case ID: TC-FUNC-OFF-001

Objective: Verify an authenticated user can retrieve all offers made on a specific property.

Preconditions: The database is running. Two users exist (Seller and Buyer). Seller owns a property. Buyer has made an offer on this property. **Test Procedure:** 1. Log in as the Seller (or Buyer, as the controller currently allows any authenticated user). 2. Navigate to the frontend view that displays offers for the specific property. 3. Alternatively, fire an

authenticated **GET** request to `/api/offer/property/:propertyId`. **Expected Result:** The server executes `getOffersByProperty`, successfully joining the `offers` and `users` tables. It returns a **200 OK** status with an array of offers that includes the `buyer_name`. The frontend successfully renders the list of offers. **Testing Type:** Functional

Test Case ID: TC-FUNC-OFF-002

Objective: Verify the "Received Offers" dashboard successfully aggregates all offers across a seller's properties. **Preconditions:** The database is running. A Seller owns multiple properties. Multiple Buyers have made offers across these different properties. The Seller is authenticated. **Test Procedure:** 1. Navigate to the frontend "Received Offers" dashboard. 2. Alternatively, fire an authenticated **GET** request to `/api/offer/received`. **Expected**

Result: The backend `getReceivedOffersHandler` extracts the Seller's ID from the JWT and executes the multi-join query (`offers -> properties -> users`). The server returns a **200 OK** status with a structured payload `{"success": true, "data": [...]}`. The data array contains all offers made on the Seller's properties, including `property_title`, `asking_price`, `buyer_name`, and `buyer_email`. **Testing Type:** Functional

Test Case ID: TC-FUNC-OFF-003

Objective: Verify unauthenticated users cannot access either offer retrieval endpoint.

Preconditions: The browser session has NO authenticated user (no JWT). A property with existing offers exists. **Test Procedure:** 1. Using Playwright's API request context, send a

GET request to `/api/offer/property/:propertyId` omitting the `Authorization` header. 2. Send a **GET** request to `/api/offer/received` omitting the `Authorization` header. **Expected Result:** The backend `authorize` middleware intercepts both requests and returns a **401 Unauthorized** status before the controllers are reached. **Testing Type:** Security / Functional

Test Case ID: TC-FUNC-OFF-004

Objective: Verify an authenticated user can successfully submit a new offer on a property. **Preconditions:** The database is running. A property exists. A user (Buyer) is authenticated. **Test Procedure:** 1. Navigate to the frontend property detail page. 2. Enter a valid numerical amount (e.g., "450000") in the offer input field and submit. 3. Alternatively, fire an authenticated **POST** request to `/api/offer` with `{ "amount": 450000, "propertyId": <id> }`. **Expected Result:** The backend `makeOfferHandler` extracts the buyer's ID from the JWT and the amount/property ID from the body. It inserts a new record into the `offers` table and returns a **201 Created** status with the new offer data. The frontend displays a success message ("Offer submitted!"). **Testing Type:** Functional

Test Case ID: TC-FUNC-OFF-005

Objective: Verify that unauthenticated users cannot submit an offer. **Preconditions:** The browser session has NO authenticated user (no JWT). A valid property exists. **Test Procedure:** 1. Using Playwright's API request context, send a **POST** request to `/api/offer` with valid payload data but no **Authorization** header. **Expected Result:** The `authorize` middleware intercepts the request and returns a **401 Unauthorized** status. The `offers` table remains unchanged. **Testing Type:** Security / Functional

Test Case ID: TC-FUNC-OFF-006

Objective: Verify the rightful property owner can successfully respond to (accept/reject) an offer. **Preconditions:** The database is running. A Seller owns a property. A Buyer has placed an offer on this property. The Seller is authenticated. **Test Procedure:** 1. Navigate to the frontend "Received Offers" dashboard. 2. Click the "Accept" or "Reject" button for the specific offer. 3. Alternatively, fire an authenticated **PATCH** request to `/api/offer/:offerId/status` with `{ "status": "ACCEPTED" }`. **Expected Result:** The `respondToOffer` controller uses `getOfferOwnership` to verify the Seller owns the linked property. It then updates the offer status in the database and returns a **200 OK** status with the updated offer. The frontend updates the UI badge to reflect the new status. **Testing Type:** Functional

Test Case ID: TC-FUNC-OFF-007

Objective: Verify unauthorized users cannot respond to an offer on a property they do not own. **Preconditions:** Three users exist: Seller, Buyer, and Attacker. Seller owns the property. Buyer made the offer. Attacker is authenticated. **Test Procedure:** 1. Log in as the Attacker. 2. Send an authenticated **PATCH** request to `/api/offer/:offerId/status` (targeting the offer made to the Seller) with `{ "status": "ACCEPTED" }`. **Expected Result:** The backend executes `getOfferOwnership`. It detects that the property's `owner_id` does not match the Attacker's JWT ID. The server immediately returns a **403 Forbidden** status with the message "Unauthorized to respond to this offer". The offer status in the database is not updated. **Testing Type:** Security / Functional